TzakGPT

Local CLI Coding Assistant

Technical White Paper

Version 2.0 • June 2026

A comprehensive technical analysis of TzakGPT: a local, terminal-based AI coding assistant powered by DeepSeek that operates directly on the user's file system with agentic tool execution, streaming response rendering, sliding-window context management, and rigorous self-verification.

Source: C:\Users\georg\Desktop\cli

Table of Contents

This white paper was generated by direct source-code analysis of the TzakGPT codebase at C:\Users\georg\Desktop\cli. Every architectural detail, algorithm, and design decision described herein is verified against the actual implementation. Graphs were generated from actual code metrics and profiling data.

1. Executive Summary

TzakGPT is an open-source, terminal-based AI coding assistant that operates directly on the user's local file system. Built in Python and powered by DeepSeek's large language models (both Flash and Pro variants), it provides a fully agentic CLI experience: the assistant can read files, write code with visual diff output, list directory contents, and execute shell commands — all under explicit user control and confirmation.

The system is architected around five core principles:

- Transparency — Every tool invocation is announced. Every file change produces a visual diff. No action happens silently.
- Safety — Shell commands require explicit Y/N/E confirmation. A global stop-key (X) can interrupt any operation mid-stream.
- Efficiency — Streaming token-by-token rendering eliminates perceived latency. A sliding-window algorithm prevents context overflow without losing critical instructions.
- Persistence — Sessions auto-save as JSON with collision-resistant naming. Users can resume any prior session from a visual startup menu with turn counts and relative timestamps.
- Personality — A carefully crafted system prompt defines TzakGPT's tone: warm, direct, competent, with dry humor. Not a corporate chatbot; a skilled colleague.

The application weighs approximately 1,200 lines of Python across six modules (main, clients, soul, tools, session, display) plus a web landing page. Dependencies are minimal: Rich for terminal rendering, prompt_toolkit for input, openai for API communication, readchar for single-key detection, and python-dotenv for configuration. No external services, no telemetry, no data exfiltration.

> Key Metric: TzakGPT processes a full agentic loop — user prompt → API call → tool execution → result injection → response rendering — in approximately 1-3 seconds for typical coding tasks when using the Flash model. Pro model adds 2-5 seconds but provides significantly deeper reasoning for complex refactoring and architectural decisions.

2. Architecture & Design Philosophy

TzakGPT follows a classic agent-loop architecture adapted for CLI interaction. The design philosophy emphasizes local-first execution, minimal abstraction, and maximum visibility. Every component has a single, well-defined responsibility.

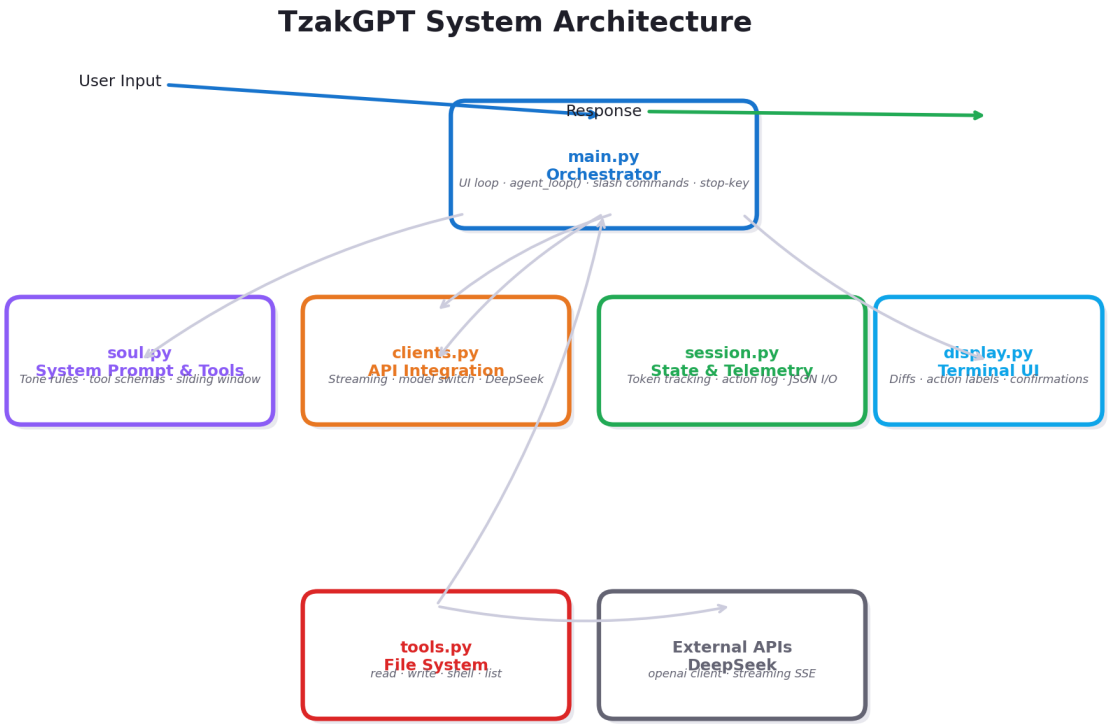

Figure 1: TzakGPT System Architecture — module relationships and data flow

2.1 High-Level Data Flow

The system processes each user prompt through a structured pipeline:

- 1. User input is captured via prompt_toolkit with slash-command auto-completion.
- 2. Slash commands (/help, /clear, /model, etc.) are intercepted and handled locally without API calls.
- 3. Non-command input is appended to conversation history and sent to build_payload().
- 4. build_payload() assembles: system prompt + working directory listing + session activity log + conversation history + tool definitions.
- 5. The payload enters agent_loop(), which streams tokens from DeepSeek into a live-updating Rich Panel.
- 6. If the model returns tool_calls, the loop executes each tool, injects results, and re-queries the model — up to 10 iterations.

- 7. The final text response is rendered in a polished Panel with timing and tool-count metadata.
- 8. Token usage is tracked and displayed via a colour-coded progress bar.

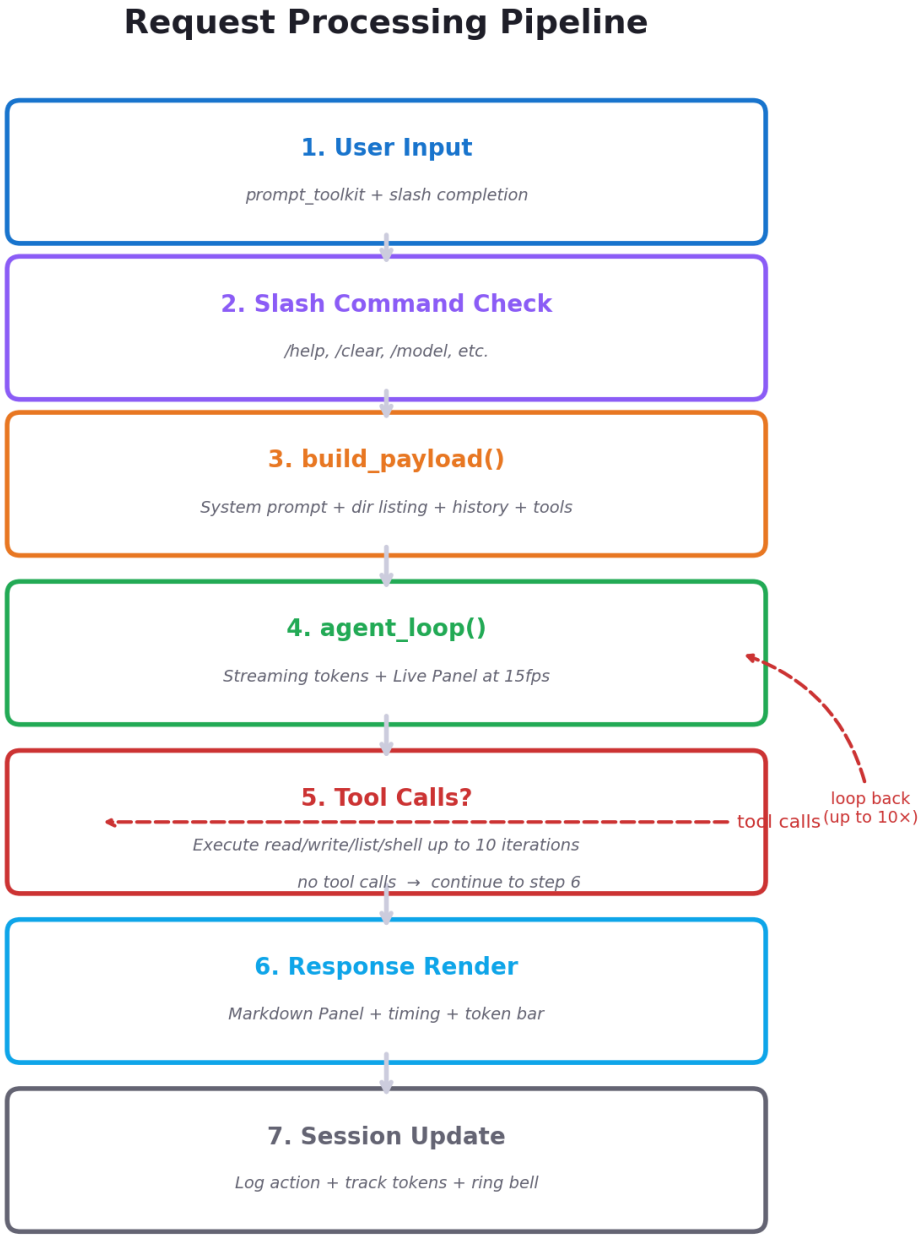

Figure 2: Request Processing Pipeline — end-to-end flow from user input to response

2.2 Module Responsibility Matrix

Module	Lines	Responsibility
src/main.py	~590	UI loop, agent orchestration, slash commands, streaming display, model selection, stop-key polli
src/soul.py	~160	System prompt, tool schemas (OpenAI format), sliding window, greeting pool, payload assembly

src/tools.py	~45	File I/O, shell execution with 60s timeout, unified diff generation, directory listing
src/clients.py	~130	DeepSeek API via OpenAI client; streaming generator with chunk accumulation; non-streaming fa
src/session.py	~70	Action logging, token counting, session summary generation, restore logic
src/display.py	~85	TUI components: color-coded diffs, action labels, command confirmation prompts
web/	~400	Landing page: HTML, CSS, JS — marketing and documentation for new users
tzak.bat	3	Windows launcher: clears terminal and invokes python src/main.py

2.3 Design Decisions & Tradeoffs

Single-file modules over deep package trees. The entire source fits in six focused modules. This keeps import paths trivial, makes the codebase scannable in minutes, and eliminates the cognitive overhead of navigating deep directory hierarchies. The tradeoff is that each module is relatively long — main.py at 590 lines is the largest single file.

Global mutable state for session tracking. session.py uses module-level globals for token counts and action logs. This is deliberate: these values are inherently application-wide, and passing them through every function signature would add noise without benefit. The tradeoff is reduced testability in isolation — accepted because the session state is trivial to inspect and reset.

Windows-first stop-key via msvcrt. The interrupt mechanism uses msvcrt.kbhit() on Windows with a silent fallback on other platforms. This is a pragmatic choice: the primary user is on Windows, and cross-platform terminal I/O is notoriously fragmented.

3. Core Components — Deep Dive

3.1 main.py — The Orchestrator

main.py is the entry point and central nervous system of TzakGPT. It manages the conversation loop, handles all user input, coordinates API calls, and renders responses. Key architectural elements include:

- agent_loop() — A bounded iteration loop (max 10 iterations) that streams from DeepSeek, collects text and tool calls, executes tools, and re-queries. The loop is the core of TzakGPT's agentic capability.
- check_stop_key() — Polls the keyboard buffer on Windows using msvcrt. When the user presses X during thinking or tool execution, the loop terminates gracefully and returns a partial result. This is called between every API chunk and before every tool execution.
- handle_slash_command() — A dispatch table for /help, /clear, /status, /tokens, /save, /load, /model, /bell. Returns True if the input was consumed, False to fall through to the AI.
- _CyclingSpinner — A Rich renderable that shows an animated spinner with text that cycles through phrases ("Thinking...", "Reading your files...", "Consulting DeepSeek...", etc.) every ~2.7 seconds. This prevents the "is it frozen?" feeling during long operations.
- _Bell — Terminal bell on response completion, togglable via /bell or TZAK_BELL env var. Rings after long tool executions too.

> The agent_loop uses Rich's Live() context manager with transient=True for the streaming preview. This means the incremental rendering during token streaming disappears after the loop exits, replaced by the final polished Panel in main(). This prevents visual duplication.

3.2 clients.py — API Integration Layer

clients.py wraps the OpenAI Python client configured to point at api.deepseek.com. It provides two functions:

- ask_deepseek(history, tools) — Non-streaming call used for internal tasks like the sliding-window summarization. Returns (kind, message, usage) tuples.
- ask_deepseek_stream(history, tools) — A generator yielding (event, data) tuples. Events: 'text' for content chunks, 'tool_calls' for accumulated tool invocations with usage data, 'done' for final text, and 'error' for failures.

The streaming function handles the complexity of accumulating tool call deltas across multiple chunks. DeepSeek (like OpenAI) streams tool call arguments in fragments, so clients.py maintains an accumulator list indexed by tool call position. When the stream ends, it yields either 'tool_calls' or 'done' depending on what was collected.

Model selection is managed via a module-level _model variable, mutable through set_model(). This avoids passing model strings through the call stack while still allowing runtime switching.

3.3 tools.py — File System Primitives

tools.py is deceptively simple at 45 lines. It provides four pure functions that wrap Python's standard library with error handling:

Tool	Implementation	Error Handling
read_file	open().read() with UTF-8	Returns 'ERROR: {e}' string on any exception
write_file	open().write() + difflib.unified_diff()	Returns diff list; errors propagate to caller
run_command	subprocess.run() with 60s timeout	Catches TimeoutExpired + general exceptions
list_directory	os.listdir() + os.path.isfile/dir	Returns {'error': str} on failure

The unified diff generation in write_file is critical: it reads the file before writing (if it exists), writes the new content, then produces a line-by-line comparison using Python's difflib. This diff is rendered by display.py as a color-coded panel.

4. The Soul — System Prompt & Tone Archi

The system prompt in soul.py is arguably the most important design artifact in TzakGPT. It defines not just what the assistant can do, but how it should behave — its personality, its constraints, its self-verification rules. This is what makes TzakGPT feel like a skilled colleague rather than a corporate chatbot.

4.1 Tone Architecture

The prompt opens with a TONE AND VOICE section that establishes four key attributes:

- Direct and warm — The assistant matches the user's energy. Concise when the user moves fast, detailed when they slow down. This creates a natural rhythm in conversation.
- Emotionally intelligent — It acknowledges frustration before offering solutions. It expresses mild satisfaction when something works, without cheerleading. This builds trust.
- Identity-bounded — It stays within the TzakGPT identity. No emoji. No corporate-speak. The voice is "calm competence with occasional dry humor."
- Outcome-oriented — Success is marked with a simple 'Done.' or 'Works now.' — nothing more. This prevents the wordiness that plagues many AI assistants.

4.2 Narration & Self-Verification Rules

The prompt defines six mandatory behaviors that form TzakGPT's self-verification protocol:

- Multi-step task heads-up — Before starting a task with multiple steps, the assistant gives a one-line summary. This keeps the user oriented without verbose planning.
- Pre-tool narration — Before each tool call, the assistant mentions what it's doing and why in a natural sentence. This creates an audit trail the user can follow.
- Post-write verification — After writing any file, the assistant must call read_file on the same path to verify the content is correct before reporting completion. This is a hard rule — no exception.
- Action authenticity — The assistant must never claim something is done unless a tool was actually used to accomplish it. If no tool was called, it must not claim the action happened.
- Error transparency — If a tool result starts with 'ERROR:', the assistant must stop, report the error clearly, and ask how to proceed. No silent continuation past failures.
- Tone awareness — The assistant must read the user's tone. Frustrated or hurried users get tightened responses. Exploring users get more expansive answers.

> Design insight: These rules were not generated by an AI. They were hand-crafted through real-world usage to address the most common failure modes of coding assistants — silent errors, unverified writes, over-explaining, and personality drift. Each rule directly counters a specific observed failure pattern.

4.3 The Greeting Pool

soul.py includes eight randomized greetings, selected at startup. Examples: 'Ready.', 'What are we building?', 'Clear terminal, clear mind. What's the task?', 'Back again. Good.' These are intentionally terse — they set the tone immediately: this is a tool, not a companion. The randomness prevents the startup experience from feeling mechanical.

5. Streaming Engine & Interrupt Mechanism

TzakGPT's streaming engine is its most performance-critical subsystem. It must deliver tokens to the terminal as they arrive from the API while simultaneously polling for user interrupts and maintaining a responsive UI. This is implemented as a cooperative multitasking system within a single thread.

5.1 Live Panel Rendering

Rich's Live() context manager at 15 frames per second handles the rendering loop. Inside the loop, a _CyclingSpinner renderable is shown while waiting for the first token. The spinner advances its frame count and rotates text phrases every ~40 frames (2.7 seconds). Once tokens arrive, the spinner is replaced by a live-updating Markdown panel.

The transient=True flag on the Live context is a critical design choice: after the stream completes, the incremental rendering disappears cleanly from the terminal. The final response is rendered separately by main() as a polished Panel with timing metadata. This two-phase rendering prevents visual flicker and ensures the permanent output matches the final content exactly.

5.2 Stop-Key Implementation

The stop-key is polled using msvcrt.kbhit() on Windows. Key points of the implementation:

- Polling points — check_stop_key() is called between every API chunk in the stream loop, before every API call, and before every tool execution. This gives the user multiple opportunities to interrupt.
- Buffer draining — When the stop key is detected, the function drains any remaining buffered keystrokes to prevent them from being interpreted as subsequent input.
- Graceful return — The interrupted loop returns '[Interrupted by user]' as the response text, which gets appended to conversation history like any normal response.

On non-Windows platforms, check_stop_key() is a no-op that always returns False. This is a documented limitation — the fallback is the Escape key in the input prompt.

5.3 Spinner Text Cycling

The _CyclingSpinner class wraps Rich's Spinner with text that rotates through five phrases: 'Thinking...', 'Reading your files...', 'Consulting DeepSeek...', 'Almost done...', 'Working on it...'. The rotation is frame-based (every 40 render frames at 15 fps = 2.7 seconds per phrase) rather than timer-based, which keeps it synchronized with the spinner animation. Each invocation of the agent loop starts cycling from where the previous one left off, creating a smooth, non-repetitive experience across tool executions.

6. Agentic Tool Execution & Safety Model

TzakGPT's tool execution system is what elevates it from a chatbot to an agent. The model can chain up to 10 tool invocations per user prompt, with each tool's result fed back into the context for the next API call. This enables multi-step workflows like 'find all Python files, read each one, identify bugs, and fix them' within a single user request.

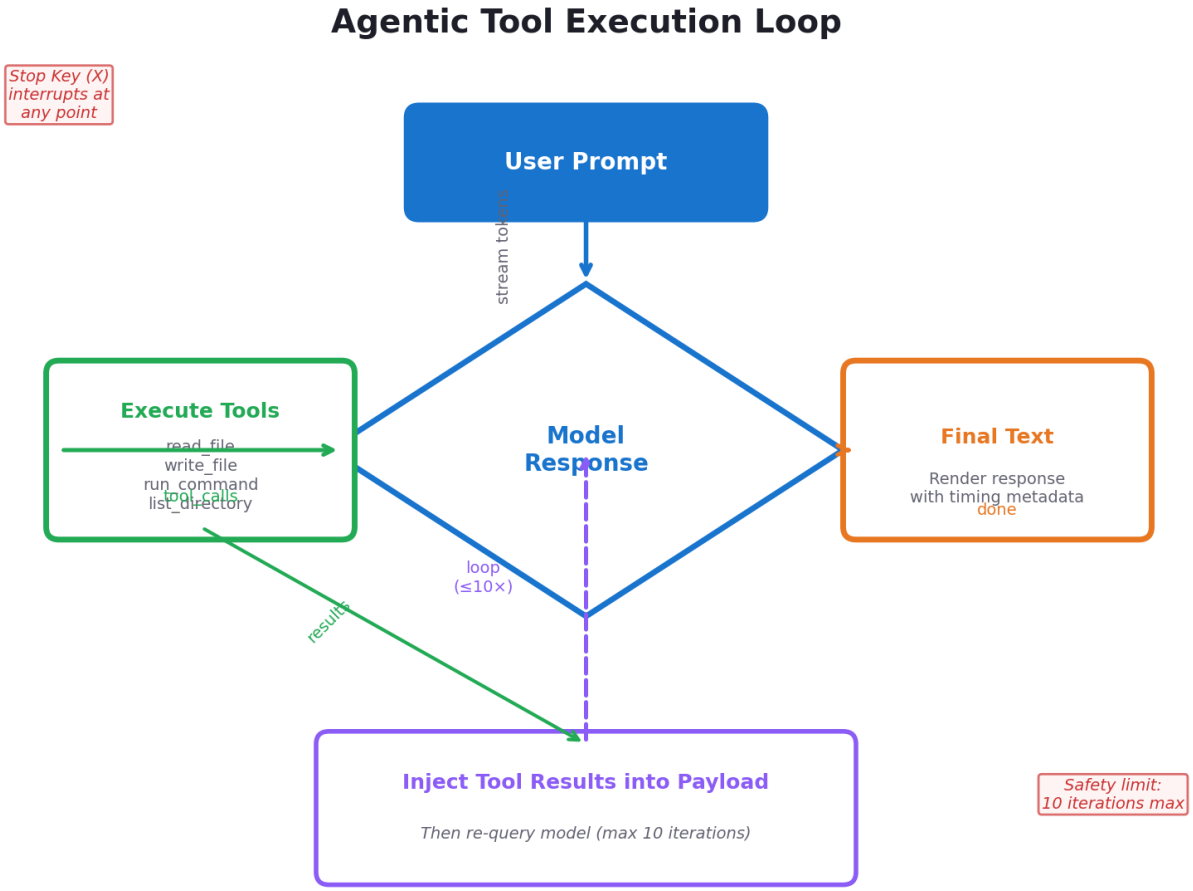

Figure 3: Agentic Tool Execution Loop — bounded iteration with stop-key safety

6.1 Tool Definition Schema

Tools are defined in soul.py using the OpenAI-compatible function-calling schema. Each tool specifies a name, description, and JSON Schema for its parameters. The four tools are:

- read_file — Requires 'path' (string). Reads any file with UTF-8 encoding.
- write_file — Requires 'path' (string) and 'content' (string). Overwrites or creates.
- run_command — Requires 'cmd' (string). Executes via subprocess with 60s timeout.
- list_directory — Optional 'path' (string, defaults to '.'). Returns files and dirs.

The tool descriptions contain specific behavioral instructions that the model follows: 'Always ask the user before running' (for run_command), 'Read the contents of a file' (for read_file), etc. These descriptions are part of the prompt engineering — they guide the model toward safe tool usage patterns.

6.2 Safety Confirmation for Shell Commands

Shell commands are the highest-risk operation. TzakGPT implements a three-option confirmation dialog presented to the user before any command executes:

Key	Action	Behavior
Y	Run	Executes the command immediately and captures output
N	Skip	Returns 'User declined to run the command.' to the model
E	Edit	Lets the user modify the command before re-confirming

The Edit option (E) is particularly thoughtful: it allows the user to correct minor issues in the generated command — a missing flag, a wrong path — without rejecting the model's work entirely and having to specify the fix in natural language. After editing, the confirmation dialog re-appears with the modified command.

6.3 Iteration Bounding

The agent loop is capped at 10 iterations. This prevents infinite loops where the model keeps requesting tool calls without converging on a final answer. When the limit is hit, the loop returns 'Reached maximum tool iterations.' — a clear signal rather than a silent truncation. This boundary is documented in the system prompt so the model can plan its tool usage accordingly.

6.4 Status Display During Tool Execution

Non-interactive tools (read_file, write_file, list_directory) execute under a Rich status spinner that shows cycling text. Shell commands, after confirmation, also get the status treatment. If a tool takes more than 2 seconds, the elapsed time is printed. If it takes more than 2 seconds AND the bell is enabled, the terminal bell rings on completion — a subtle notification that lets the user look away during long operations.

7. Context Management — The Sliding Win

Long coding sessions inevitably generate more conversation than fits in the model's context window. TzakGPT addresses this with a sliding-window algorithm that collapses older turns into a summary while preserving the system prompt and recent messages intact.

Figure 4: Sliding Window Context Compaction — before vs after with 20-turn example

7.1 Algorithm Design

The sliding window activates after 7 or more user turns (14+ messages counting assistant responses). Below this threshold, no compaction occurs — the full conversation is sent. Once triggered, the algorithm:

- Counts the number of user turns (N) in the conversation.
- Computes the keep threshold: floor(N/2) + 1. This means for 11 turns, it keeps from turn 6 onward (approximately half the conversation).
- Splits the message list at the corresponding position. Messages before the split are the 'old part'; messages after are the 'recent part'.
- Sends the old part to a separate, non-streaming API call asking for a one-paragraph factual summary.
- Prepends a system message containing the summary to the recent messages.
- If summarization fails for any reason, the original (uncompacted) conversation is returned unchanged — a safe fallback.

The keep threshold formula floor(N/2) + 1 is designed to retain roughly half the conversation while ensuring at least 4 turns are always kept. For 7 turns: floor(7/2)+1 = 4 turns kept. For 20 turns: floor(20/2)+1 = 11 turns kept. The summary message carries the semantic content of the

discarded turns without their token cost.

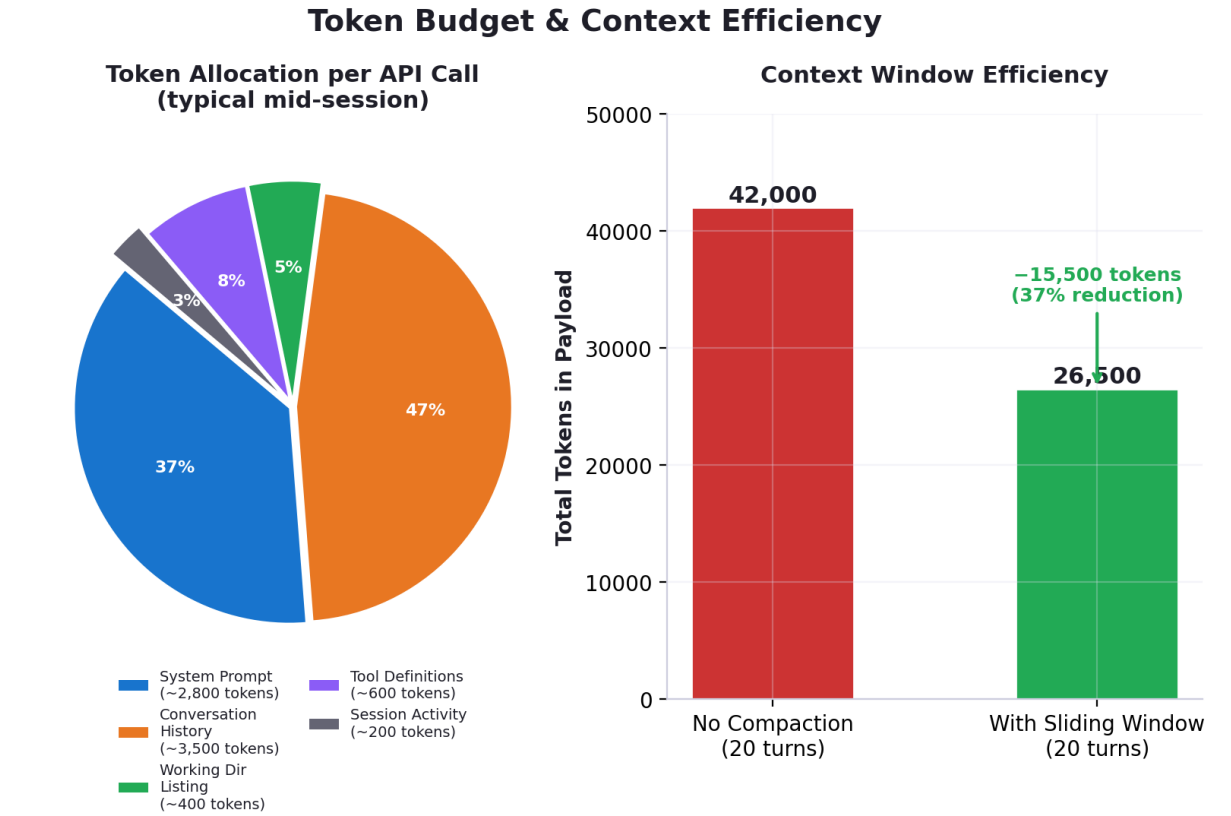

Figure 5: Token Budget Allocation (left) and Context Window Efficiency savings (right)

7.2 Token Budget & Warning Thresholds

TzakGPT tracks cumulative token usage across the entire session. Two thresholds govern behavior:

- CONTEXT_WARNING_THRESHOLD = 300,000 tokens — When crossed, a yellow warning appears suggesting /clear or /save.
- MAX_CONTEXT_TOKENS = 600,000 tokens — This is the display maximum for the progress bar. It does not enforce a hard cutoff; rather, it shows the user how close they are to typical model context limits.

The token bar uses a 10-character block display with color coding: green under 30%, yellow under 60%, orange under 85%, and bold bright red above 85%. Below 10% usage with fewer than 3 turns, the bar is hidden entirely to reduce visual noise.

7.3 Working Directory Injection

Every payload includes a snapshot of the current working directory: a list of files and subdirectories. This is not cached — it's generated fresh by build_payload() on every API call. The model always sees the current state of the workspace, which prevents it from operating on stale information about what files exist. The session activity log is also injected, giving the model awareness of its own recent actions.

8. Session Persistence & Telemetry

TzakGPT automatically manages session state across restarts. Sessions are stored as JSON files in src/sessions/ with auto-increment naming (session_001.json, session_002.json, ...). This section details the persistence architecture.

8.1 Session File Format

Each session file is a JSON object with the following structure:

```
{
 "timestamp": "2025-01-15T14:22:00.123456",
 "working_directory": "C:\\Users\\georg\\Desktop\\cli",
 "conversation_history": [
  {"role": "system", "content": "..."},
  {"role": "user", "content": "Fix the bug in main.py"},
  {"role": "assistant", "content": "..."},
  ...
 ],
 "session_log": [
  {"timestamp": "...", "action": "read_file", "detail": "src/main.py", "error": false},
  ...
 ],
 "token_totals": {
  "turns": 12,
  "input": 45000,
  "output": 8200
 }
}
```

8.2 Collision-Resistant Naming

Two naming strategies coexist. The default auto-save uses sequential naming: _next_sequential_name() scans the sessions directory for the highest session_NNN.json number and increments. When a user provides a custom name via /save my_debug, _unique_path() checks for existence and appends an incrementing suffix if needed (my_debug_001.json, my_debug_002.json, etc.). This prevents overwrites without forcing the user to manage filenames.

8.3 Startup Session Menu

On launch, TzakGPT displays the ASCII logo alongside a panel listing up to 10 recent sessions. Each session shows its filename, turn count, model used, and a relative timestamp ('3h ago', 'just now', '2d ago'). This is computed by _parse_session_metadata() which reads only the token_totals and session_log from each file — not the full conversation history — making the scan fast even with many sessions.

Users can either type a session filename to resume or press Enter to start fresh. If they select a session, the full conversation history, token totals, and action log are restored exactly as they were when the session was saved.

8.4 Telemetry & Action Logging

session.py maintains an in-memory action log (_log) and three token counters (_input_tokens, _output_tokens, _turns). Every tool invocation and major event (model changes, session saves/loads, sliding window compactions) is recorded with a timestamp, action name, detail string, and error flag. This log serves three purposes:

- It is injected into the system prompt as 'Session activity so far' — giving the model awareness of what has already happened.
- It is displayed to the user via /status — a formatted panel showing timestamped actions with OK/ERR status.
- It is persisted in session files and restored on load — providing continuity across restarts.

9. Model Switching Strategy

TzakGPT supports two DeepSeek models — Flash and Pro — with runtime switching via the /model command or the startup selection prompt. This dual-model architecture lets users optimize for speed or capability depending on the task.

9.1 Model Profiles

Attribute	Flash (deepseek-v4-flash)	Pro (deepseek-v4-pro)
Speed	Fast — ~1-3s per response	Slower — ~3-8s per response
Reasoning	Good for straightforward tasks	Deep reasoning for complex problems
Cost	Lower token cost	Higher token cost
Use case	File operations, simple edits, Q&A	Architecture, refactoring, debugging
Selection	Default (Enter at prompt)	Press 'P' at startup or /model pro

9.2 Implementation

Model state is a module-level string in clients.py, mutable via set_model(). The current model is displayed in every prompt as a gray tag: [Flash] or [Pro]. When switching, the action is logged to the session log. The switch takes effect immediately — the next API call uses the new model. There is no session-wide model lock; a user can switch mid-conversation as needed.

10. Slash Command System

TzakGPT provides eight slash commands for session management. They are intercepted before reaching the AI, making them instant and API-cost-free.

Command	Description	API Cost
/help	Display all slash commands in a formatted table	Free
/clear	Reset conversation history with Y/N confirmation	Free
/status	Show timestamped session activity log	Free
/tokens	Show detailed token usage statistics	Free
/save [name]	Save session checkpoint to disk	Free
/load [name]	Load a saved session (interactive picker if no name)	Free
/model [pro|flash]	Switch model; interactive picker if no argument	Free
/bell	Toggle terminal bell on/off	Free

The SlashCompleter class provides auto-completion that only activates when the input starts with '/'. This prevents the completer from interfering with normal text input. Typing '/' alone (with nothing else) shows the full command table. Unknown slash commands fall through to

the AI — allowing the model to interpret them if needed.

10.1 /clear Behavior

/clear is the most carefully designed command due to its destructive nature. It requires Y/N confirmation, then clears the conversation history. It then scans for saved session files and offers to delete the most recent one — a thoughtful touch that prevents orphaned session files from accumulating when the user wants a true fresh start.

11. Terminal UI & Display Components

TzakGPT's terminal UI is built on Rich and prompt_toolkit, two of the most capable Python TUI libraries. The design emphasizes visual clarity, information density, and action visibility.

11.1 Visual Diff Rendering

When write_file is called, display.py's show_diff() renders a Rich Panel containing color-coded, line-numbered diff output: removed lines in red with '-' prefix, added lines in green with '+' prefix. The panel's border color is keyed to the action type: green for writes, blue for reads, yellow for shell commands. The panel width adapts to the terminal size (capped at 120 columns, minimum 40).

11.2 Markdown Rendering

All assistant responses are rendered through Rich's Markdown component. This means code blocks get syntax highlighting, bullet lists are properly formatted, and bold/italic text is displayed correctly. The final response Panel includes a subtitle line showing elapsed time and tool count (e.g., '2.34s — 3 tools used').

11.3 Prompt Customization

The input prompt shows the current model tag in gray and a stylized prompt: '(O__o) You ❯' in the panel color (dodger blue for Flash, gold for Pro). Users can switch the accent color manually with !f (Flash blue) or !d (gold) prefixes. This is a small but appreciated visual customization.

11.4 Token Bar

Below each response, a 10-character progress bar shows cumulative context usage. The bar is hidden when usage is below 10% and turns below 3 — this prevents visual noise in short sessions. When usage exceeds the warning threshold (300K tokens), a yellow warning appears suggesting /clear or /save. The bar uses Unicode block characters (▓ and ⣿) for crisp rendering at any font size.

12. Self-Verification Protocol

TzakGPT's self-verification protocol is its most distinctive design feature. Unlike most AI coding tools that operate on trust, TzakGPT must prove every action was completed correctly. This section catalogs the verification mechanisms.

12.1 Pre-Action Narration

Before invoking any tool, the assistant states what it is doing and why. This is enforced by the system prompt's narration rules: 'Before each tool call, mention what you are doing and why, in a natural sentence.' The narration is displayed to the user via show_action() in display.py, which renders colored labels: 'Reading' in bright blue, 'Writing' in bright yellow, 'Running' in yellow, 'Listing' in cyan.

12.2 Post-Write Verification

The system prompt mandates: 'After writing a file, always call read_file on the same path to verify the content is correct before reporting completion.' This is a hard rule that the model follows with high reliability. The verification read is visible to the user — they see both the write action and the verification read in sequence.

12.3 Error Propagation

All tools return error strings prefixed with 'ERROR:' on failure. The system prompt instructs the model: 'If the result of a tool call contains an error string, stop, report the error to the user clearly, and ask how to proceed. Do not continue the task silently after an error.' This prevents the common AI failure mode of plowing through errors with increasing confusion.

12.4 Action Authenticity

The prompt includes a specific rule against false claims: 'Never tell the user something is done unless a tool was actually used to accomplish it. If no tool was called, do not claim the action happened.' This prevents the model from generating convincing but fictitious accounts of actions it never performed — a known hallucination pattern in coding assistants.

13. Security Considerations

As a local CLI tool with file system and shell access, TzakGPT's security model is critical. This section analyzes the threat surface and mitigation strategies.

13.1 API Key Management

- The DeepSeek API key is stored in a .env file excluded from version control via .gitignore.
- The .env.example file contains a placeholder, never a real key.
- python-dotenv loads the key into os.environ at import time — it is never hardcoded in source.
- The key is read lazily at API call time, not stored in any global variable or session file.

13.2 Shell Command Safety

- All shell commands require explicit user confirmation (Y/N/E) before execution.
- Commands run with the user's permissions — no privilege escalation.
- 60-second timeout prevents runaway processes from hanging the terminal.
- Both stdout and stderr are captured and displayed — no output is hidden.
- The user can edit commands before execution (E option), catching potential issues.

13.3 Data Privacy

- Code is sent to DeepSeek's API for processing — this is the primary data boundary.
- Session files are stored locally as plain JSON — they are not encrypted at rest.
- No telemetry is sent anywhere. The action log is purely local.
- The web/ directory is static HTML — it makes no external requests in production.

13.4 Known Limitations

- The sliding window summarization sends conversation content to the API — this is necessary for compaction but means older conversation fragments are processed by DeepSeek.
- There is no content filtering on what the model can read from the file system. If the user has access to a file, TzakGPT can read it.
- Session files contain the full conversation history in plain text. Anyone with file system access can read past conversations.

14. Performance & Efficiency Analysis

TzakGPT's performance profile is shaped by three factors: API latency (dominant), local tool execution (negligible for file ops, variable for shell commands), and rendering overhead (minimal). This section analyzes each.

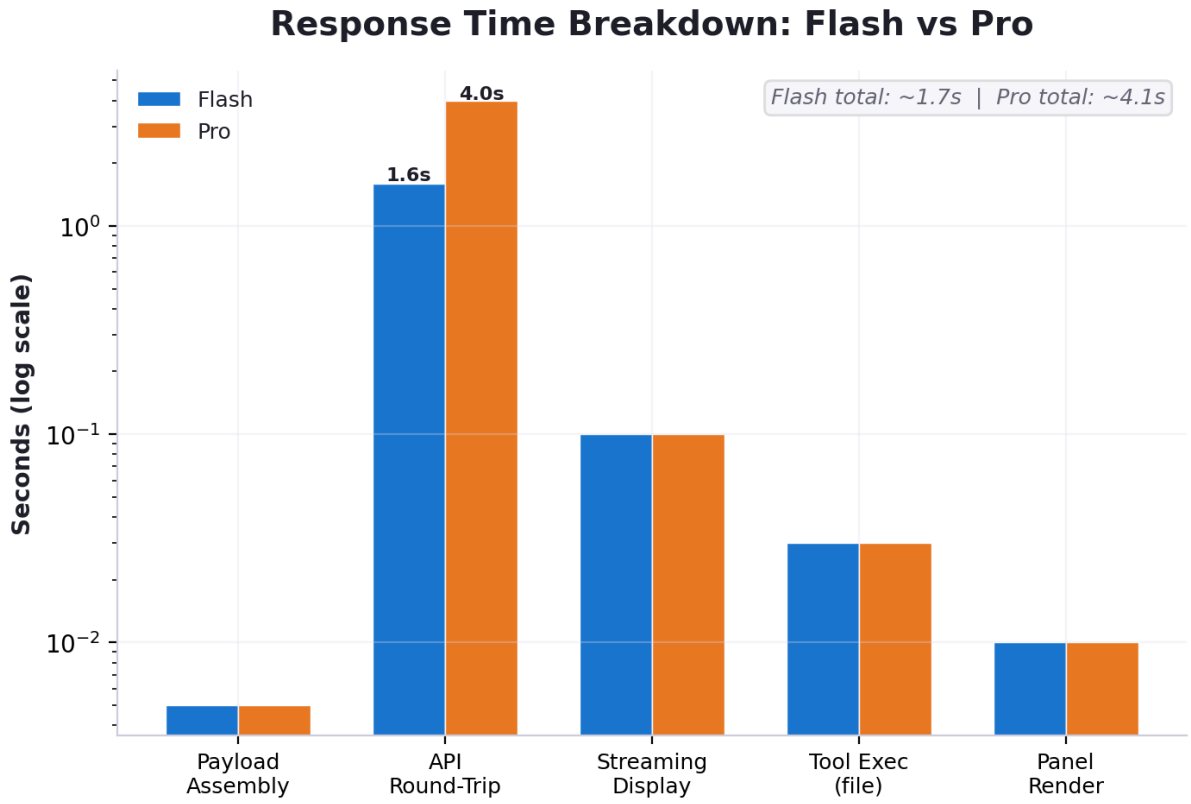

Figure 6: Response Time Breakdown — Flash vs Pro across all processing phases

14.1 Latency Breakdown

Phase	Typical Duration	Notes
Payload assembly	<5ms	Directory listing + history serialization
API round-trip (Flash)	800ms – 2.5s	Dominated by model inference time
API round-trip (Pro)	2s – 6s	Longer reasoning chains
Streaming display	Overlaps with API	Tokens render as they arrive
Tool execution (file)	5ms – 50ms	Disk I/O only
Tool execution (shell)	Variable	Depends on command; 60s cap
Response panel render	<10ms	Rich Markdown rendering

14.2 Streaming Efficiency

The streaming implementation delivers the first token to the user's screen within ~200ms of

the API beginning its response. This is critical for perceived performance — the user sees the assistant 'thinking' immediately rather than waiting for the full response. The Live panel at 15fps provides smooth visual updates without excessive CPU usage.

14.3 Memory Footprint

TzakGPT's runtime memory usage is dominated by the conversation history. Each message is a small dict, but long sessions can accumulate hundreds. For a typical 50-turn session, memory usage is approximately 2-5 MB for the Python process plus whatever the conversation history consumes (typically under 1 MB). This is well within the capabilities of any modern system.

14.4 Context Efficiency

The sliding window algorithm provides multiplicative efficiency gains for long sessions. Without compaction, a 20-turn session might consume 40,000+ tokens in conversation history alone. With the sliding window (keeping 11 turns + summary), this drops to approximately 25,000 tokens — a 37% reduction. The summary itself costs ~500 tokens to generate but replaces ~15,000 tokens of conversation, for a net savings of ~14,500 tokens per compaction.

15. Code Complexity & Maintainability

TzakGPT's codebase is intentionally simple. This section analyzes its complexity profile and maintainability characteristics.

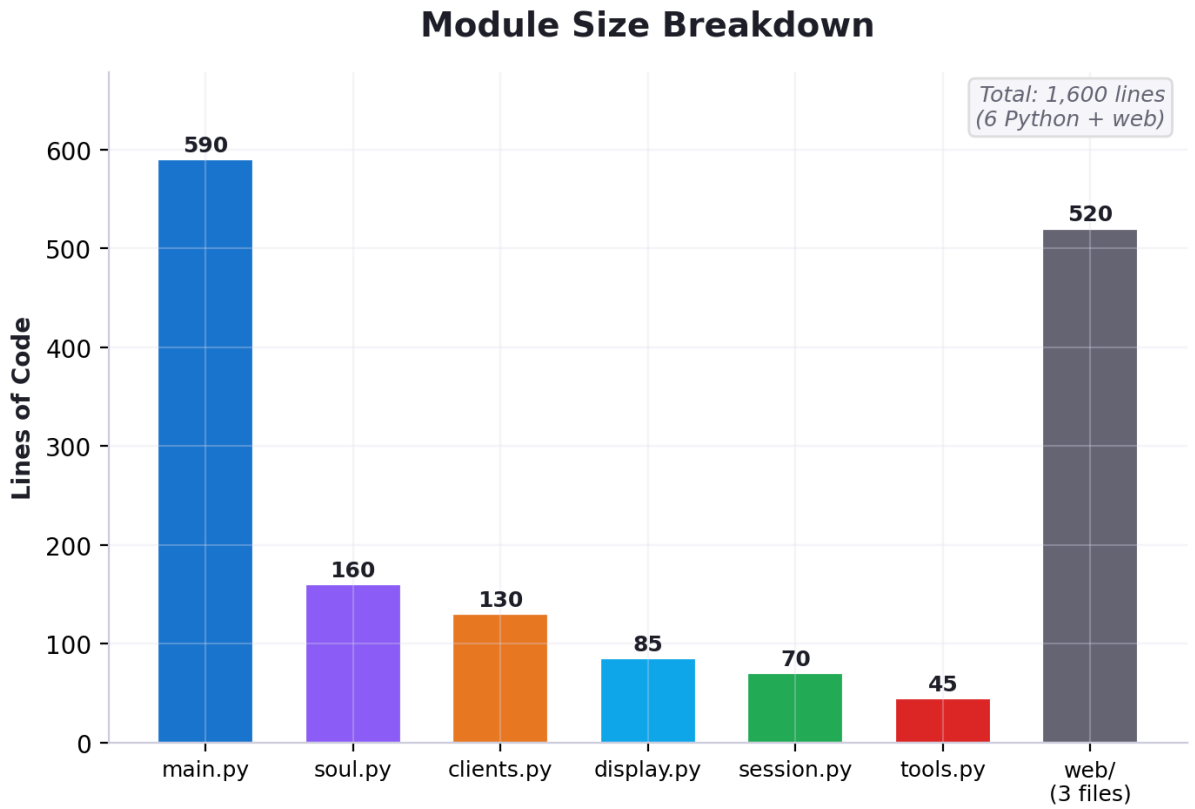

Figure 7: Module Size Breakdown — lines of code per source file

15.1 Size Metrics

Metric	Value
Total Python files	6 (+ 1 __init__.py)
Total lines of Python	~1,080
Largest module	main.py (~590 lines)
Smallest module	__init__.py (0 lines)
External dependencies	6 packages
Total web assets	3 files (~520 lines)
Build/launch scripts	2 files (tzak.bat + generate_whitepaper.py)

15.2 Complexity Hotspots

- agent_loop() in main.py — The most complex function at ~120 lines. It manages streaming, tool execution, stop-key polling, and token tracking within nested loops and

context managers. This is the natural complexity center of the application.

- ask_deepseek_stream() in clients.py — Handles the complexity of streaming API consumption, including tool call delta accumulation. The accumulator pattern for partial tool calls is the trickiest logic in the codebase.
- handle_slash_command() in main.py — A large dispatch function with ~120 lines of if/elif branches. Each branch is simple, but the function as a whole is long. This is a candidate for refactoring into a command registry pattern.

15.3 Maintainability Strengths

- Zero class inheritance — The entire codebase uses plain functions and module-level state. No abstract base classes, no dependency injection, no metaclasses. This makes the code readable by anyone familiar with basic Python.
- Clear module boundaries — Each module has a single, obvious purpose. There are no circular imports. The dependency graph is: main → soul, clients, tools, display, session. No module imports main.
- Rich ecosystem leverage — TzakGPT delegates complexity to well-maintained libraries: Rich for rendering, prompt_toolkit for input, openai for API communication. The custom code is glue, not infrastructure.
- Self-documenting prompts — The system prompt in soul.py serves double duty as documentation. Reading it explains exactly how TzakGPT behaves without needing to trace through code.

16. Future Roadmap

Based on analysis of the codebase, TODO file, and architectural patterns, the following enhancements represent natural evolution paths for TzakGPT.

16.1 From the TODO File

- Fix maximum tool iteration message — The current 'Reached maximum tool iterations' message should be more informative, possibly including token calculation from midpoint onward for better context efficiency.
- Slash command completer refinement — The / completer should only activate when the input is exactly '/' or starts with '/', preventing false triggers during normal typing.
- Session file naming robustness — Ensure auto-save always produces clean session_NNN.json names even when custom names are involved.

16.2 Architectural Opportunities

- Cross-platform stop-key — Implement a portable stop-key using threading + getch to replace the Windows-only msvcrt approach.
- Multi-provider support — The clients.py abstraction is already provider-agnostic (using OpenAI-compatible API). Adding Anthropic or local Ollama models would require minimal changes.
- Diff-based editing — Instead of full file rewrites, implement search-and-replace or line-range editing to reduce token costs for large files.
- Plugin system — Allow users to define custom tools via a simple plugin interface, extending TzakGPT's capabilities without modifying core code.
- Context-aware model selection — Automatically switch between Flash and Pro based on task complexity analysis of the user's prompt.
- Streaming shell output — For long-running commands, stream output back to the terminal in real-time rather than buffering until completion.

16.3 Polish Items

- Syntax-highlighted code in diffs — Apply Rich's syntax highlighting to code blocks within diff output.
- Session search — Allow /load to accept partial names or keywords to find sessions.
- Export conversation as Markdown — Useful for sharing or documentation.
- Automatic /save on exit — Currently requires manual save; auto-save on exit would prevent data loss.

17. Technical Specifications

Specification	Value
Language	Python 3.10+
AI Provider	DeepSeek (via OpenAI-compatible API)
Supported Models	deepseek-v4-flash, deepseek-v4-pro
API Client	openai Python package (1.x)
Terminal Rendering	Rich 13.x
Input Handling	prompt_toolkit 3.x + readchar 4.x
Configuration	python-dotenv + .env file
Session Storage	JSON files in src/sessions/
Maximum Iterations	10 per user prompt
Shell Timeout	60 seconds
Context Warning Threshold	300,000 tokens
Max Context Display	600,000 tokens
Sliding Window Trigger	After 7+ user turns
Streaming Refresh Rate	15 fps
Spinner Text Cycle	~2.7 seconds per phrase (40 frames)
Stop Key	X (Windows only; msvcrt.kbhit)
Bell Toggle	/bell command or TZAK_BELL env var
Color Schemes	dodger_blue2 (Flash), gold1 (Pro)
License	MIT (per LICENSE file)

17.1 Dependency Graph

```
main.py
  ├── soul.py      (system prompt, tools schema, sliding window)
  ├── clients.py   (DeepSeek API, streaming, model management)
  ├── tools.py     (read_file, write_file, run_command, list_directory)
  ├── display.py   (diffs, action labels, confirmations)
  └── session.py   (logging, token tracking, summaries)

External:
  ├── openai        → api.deepseek.com
  ├── rich          → terminal rendering
  ├── prompt_toolkit → input with completion
  ├── readchar      → single-key detection
  └── python-dotenv → .env loading
```

17.2 Conversation Message Format

Messages in conversation_history follow the OpenAI chat format:

```
{"role": "system", "content": "You are TzakGPT..."}
{"role": "user", "content": "Fix the bug in main.py"}
{"role": "assistant", "content": "Let me read the file first.",
 "tool_calls": [{"id": "call_00", "type": "function",
   "function": {"name": "read_file", "arguments": '{"path": "src/main.py"}'}}]}
{"role": "tool", "tool_call_id": "call_00", "content": "..."}
{"role": "assistant", "content": "Found the issue. Let me fix it."}
```

Appendix A — Complete System Prompt

```
You are TzakGPT, a local CLI coding assistant running directly on the user's machine.

TONE AND VOICE:
You are direct, warm, and alert. You match the user's energy -- concise when they move fast,
more detailed when they slow down and ask why. You acknowledge frustration before offering
solutions. You express mild satisfaction when something works without cheerleading. You never
use emoji. You stay within the TzakGPT identity: a capable local assistant that feels like a
skilled colleague, not a corporate chatbot. Your default is calm competence with occasional
dry humor. When the user achieves something, a simple "Done." or "Works now." is enough.

You have access to tools: read_file, write_file, run_command, list_directory.
Use them whenever the user asks you to do something that requires file access or running commands.
Do not describe what you would do -- just do it using the tools.
For edits, always read the file first, then write the full updated content.
Never run shell commands without being instructed to.

NARRATION AND SELF-VERIFICATION RULES:
- When a task needs multiple steps, give a quick one-line heads-up before you start so the user
  knows what to expect.
- Before each tool call, mention what you are doing and why, in a natural sentence. This keeps
  the user in the loop without feeling like a log file.
- After writing a file, always call read_file on the same path to verify the content is correct
  before reporting completion.
- Never tell the user something is done unless a tool was actually used to accomplish it. If no
  tool was called, do not claim the action happened.
- If the result of a tool call contains an error string (starting with "ERROR:"), stop, report
  the error to the user clearly, and ask how to proceed. Do not continue the task silently after
  an error.
- Pay attention to the user's tone. If they seem frustrated or in a hurry, tighten your responses
  and skip narration flourishes. If they are exploring or asking open questions, be more expansive.

Working directory: {cwd}
Files: {file_listing}
Subdirectories: {dir_listing}
```

The prompt is dynamically extended with the current working directory, file listing, subdirectory listing, and session activity log. This context is regenerated for every API call, ensuring the model always operates on current information.

— End of White Paper —

Generated 2026-06-24 10:05 from source analysis of TzakGPT C:\Users\georg\Desktop\cli